

Object Oriented Programming (CSC166)

Lab Sheet 6

1. Write a program that defines a base class "Shape" with a virtual function area() that returns 0. Derive two classes "Circle" and "Rectangle" from Shape that override area() to compute their respective areas. Use a base class pointer to call the overridden function on each derived object and demonstrate late binding.

Hint: [virtual float area() → virtual function in Shape, returns 0

*Circle(float r) → parameterized constructor; overridden area() returns $3.1416 * r * r$*

*Rectangle(float l, float w) → parameterized constructor; overridden area() returns $l * w$*

Use Shape ptr = &circleObj; ptr->area(); and Shape* ptr2 = &rectObj; ptr2->area(); to demonstrate late binding through a base class pointer]*

2. Write a program that defines an abstract class "Animal" with a pure virtual function makesound(). Derive two classes "Dog" and "Cat" from Animal that implement makesound() differently. Use base class pointers to call the correct overridden function for each derived object.

Hint: [virtual void makesound() = 0; → pure virtual function that makes Animal an abstract class

Dog::sound() → prints "Dog barks"

Cat::sound() → prints "Cat meows"

Use Animal ptr = new Dog(); ptr->makesound(); and Animal* ptr2 = new Cat(); ptr2->makesound(); to call the derived versions*

Give Animal a virtual destructor so that deleting through a base pointer correctly calls the derived class destructor]

3. Write a program that defines a class "Account" with a private data member balance and a static data member totalAccounts that keeps track of how many Account objects have been created. Use a friend function to add interest to the balances of two different Account objects at once.

Hint: [Account(float bal) → parameterized constructor that increments totalAccounts

static int totalAccounts; → static data member shared by all objects

static int getCount() → static member function that returns totalAccounts

friend void addInterest(Account &a1, Account &a2, float rate) → friend function that adds rate% interest to the balance of both a1 and a2

displayBalance() → prints "Balance: balance"]